



Online Food Ordering Assistant

A Case Study in LLM Function Calling

Assignment: Function Calling

Architecture, LLM Reasoning Loop, Tool Schemas & Worked Example

Prepared using: Claude Sonnet 5 (Anthropic) — function calling / tool-use paradigm

Date: August 2026

1. Introduction

Ordering food online today typically means tapping through several rigid screens: search, filter, menu, cart, checkout, coupon field, payment. A conversational assistant can collapse this into a single natural-language exchange — “Order me a large Margherita from Domino’s and use my SAVE20 coupon” — but only if the underlying Large Language Model (LLM) can do more than talk. It has to actually search a restaurant database, place a real order, apply a real coupon and trigger a real payment.

This is exactly the gap that Function Calling (also called Tool Use) closes. This document explains what function calling is, designs the architecture of an Online Food Ordering Assistant built around it, defines the tool schemas the LLM can call, walks through the end-to-end request/response loop with diagrams, and discusses the benefits and limitations of the approach.

1.1 Objective

- Explain the concept of function calling in modern LLMs.
- Design a layered system architecture for a food-ordering assistant.
- Define concrete function/tool schemas the assistant can invoke.
- Trace a full conversation through the function-calling loop with a sequence diagram.
- Summarize benefits, limitations and best practices.

2. What Is Function Calling in LLMs?

By default, an LLM can only generate text — it has no way to check today’s menu, look up an order status, or charge a card. Function calling (tool use) is a protocol that lets a developer describe a set of available functions to the model — their names, purposes and expected arguments — using a structured schema (typically JSON Schema). During a conversation, instead of only replying in prose, the model can respond with a structured request to invoke one of those functions with specific argument values.

The application (not the LLM itself) is responsible for actually executing the function — querying a database, calling a payment API, hitting a delivery-tracking service — and then feeding the function's result back to the model as a new message. The model reasons over that result and either calls another function or produces a final natural-language answer for the user.

2.1 Core Idea in One Sentence

Function calling turns an LLM from a text generator into an orchestrator that can decide when, which, and how to invoke external tools — while the actual side effects (reads/writes to real systems) always happen in code the developer controls, never inside the model.

2.2 Anatomy of a Function Schema

Every function exposed to the model is described declaratively, for example:

```
{
  "name": "place_order",
  "description": "Places a food order for the given restaurant, items and payment method.",
  "input_schema": {
    "type": "object",
    "properties": {
      "restaurant_id": { "type": "string" },
      "items": {
        "type": "array",
        "items": {
          "type": "object",
          "properties": {
            "item_name": { "type": "string" },
            "size": { "type": "string", "enum": ["S", "M", "L"] },
            "quantity": { "type": "integer", "minimum": 1 }
          }
        },
        "required": ["item_name", "quantity"]
      },
      "coupon_code": { "type": "string" },
      "payment_method": { "type": "string", "enum": ["card", "upi", "wallet", "cod"] }
    },
    "required": ["restaurant_id", "items", "payment_method"]
  }
}
```

```
}  
  
}
```

The LLM never sees, and never needs to see, how `place_order()` is implemented — only its name, description and parameter contract. This is what makes tool use portable across completely different backends.

3. Use Case: The Online Food Ordering Assistant

The assistant supports the full conversational ordering journey:

1. Discover restaurants and menu items near the user.
1. Build and modify a cart (add / remove / resize items).
2. Apply coupons and check delivery fees or minimum-order rules.
3. Place the order and initiate payment.
4. Track an order already in progress, or request a cancellation/refund.

Each of these journeys maps onto one or more callable functions, described in Section 6.

4. System Architecture

The system is organized into five layers. The client layer captures user intent; the orchestrator manages conversation and session state; the LLM reasons about intent and emits structured function calls; the function/tool layer executes those calls against real backend systems; and the external systems layer holds the actual restaurant, order, payment and delivery data.

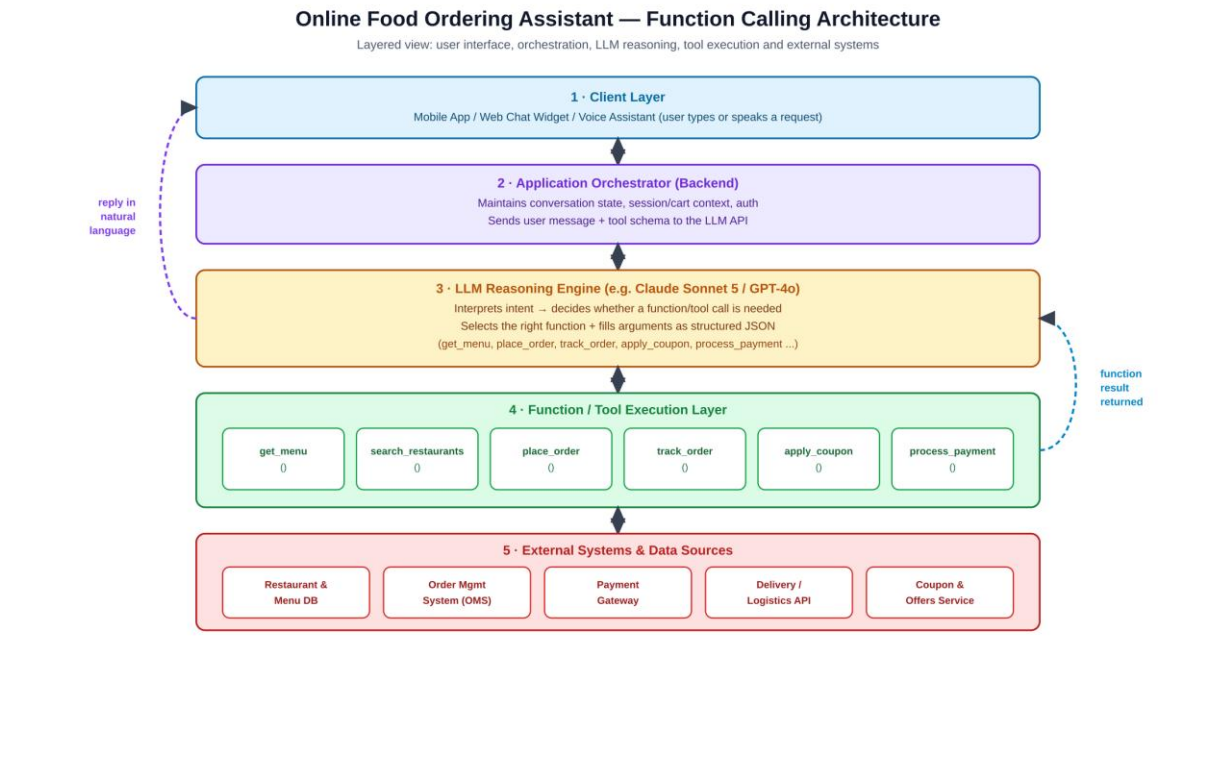


Figure 1 — Layered architecture of the Online Food Ordering Assistant

4.1 Layer Responsibilities

Layer	Responsibility	Example Technology
Client	Captures the user's text/voice request and renders the assistant's reply	React Native app, web chat widget
Orchestrator	Holds conversation history, cart/session state, auth token; calls the LLM API and executes tool calls	Node.js / Python backend service
LLM Engine	Interprets intent, selects a function, fills its arguments, and drafts the final reply	Claude Sonnet 5 (function calling / tool use)
Function Layer	Concrete implementations of each tool: validates arguments, calls internal services	Python/TypeScript service functions
External Systems	Systems of record: menu DB, order management, payment gateway, delivery API	PostgreSQL, Stripe/Razorpay,

Layer	Responsibility	Example Technology
		logistics API

5. LLM Used and Why

This design uses Claude Sonnet 5 as the reasoning engine, accessed through Anthropic's Messages API with a tools parameter carrying the JSON-Schema function definitions from Section 6. Any modern function-calling-capable model (e.g. GPT-4o, Gemini 1.5/2.x) could be substituted with the same architecture, since the tool-use pattern itself is model-agnostic — only the request/response wire format differs slightly.

5.1 Why This Class of Model Fits the Task

- Reliable structured output: emits well-formed JSON arguments matching the declared schema.
- Multi-step tool chaining: can call `search_restaurants()`, then `place_order()`, then `track_order()` across turns without losing context.
- Strong natural-language grounding: turns raw JSON results (prices, ETAs) back into a friendly, readable confirmation.
- Refusal / clarification behaviour: asks a follow-up question instead of guessing when a required argument (e.g. delivery address) is missing.

5.2 Model Configuration Snapshot

Setting	Value
Model	claude-sonnet-5
Mode	Tool use / function calling (tools parameter)
Tool choice	auto (model decides if/which function to call)
Max tool-call turns per request	5 (safety cap to avoid infinite loops)
Temperature	0.2 (favours deterministic argument extraction)

6. Function / Tool Definitions

The orchestrator exposes the following six tools to the LLM. Each is intentionally narrow and single-purpose — a best practice that keeps the model's tool choice unambiguous.

Function	Purpose	Key Parameters
search_restaurants	Finds restaurants by cuisine, name or location	query, cuisine, lat, lng, max_distance_km
get_menu	Retrieves the live menu for a chosen restaurant	restaurant_id
apply_coupon	Validates a coupon and returns the discounted total	cart_id, coupon_code
place_order	Creates the order, charges payment, confirms with the OMS	restaurant_id, items[], coupon_code, payment_method
track_order	Returns live status and ETA for an existing order	order_id
cancel_or_refund_order	Cancels an order or requests a refund	order_id, reason

6.1 Example Schema — track_order

```
{
  "name": "track_order",
  "description": "Returns current status, courier location and ETA for an order.",
  "input_schema": {
    "type": "object",
    "properties": {
      "order_id": { "type": "string", "description": "Order confirmation ID, e.g. ORD-48213" }
    },
    "required": ["order_id"]
  }
}
```

```

}
}

```

7. Function Calling Workflow

The diagram below traces one complete user request through every layer of the architecture, including two chained function calls (`search_restaurants` then `place_order`) before the model composes its final reply.

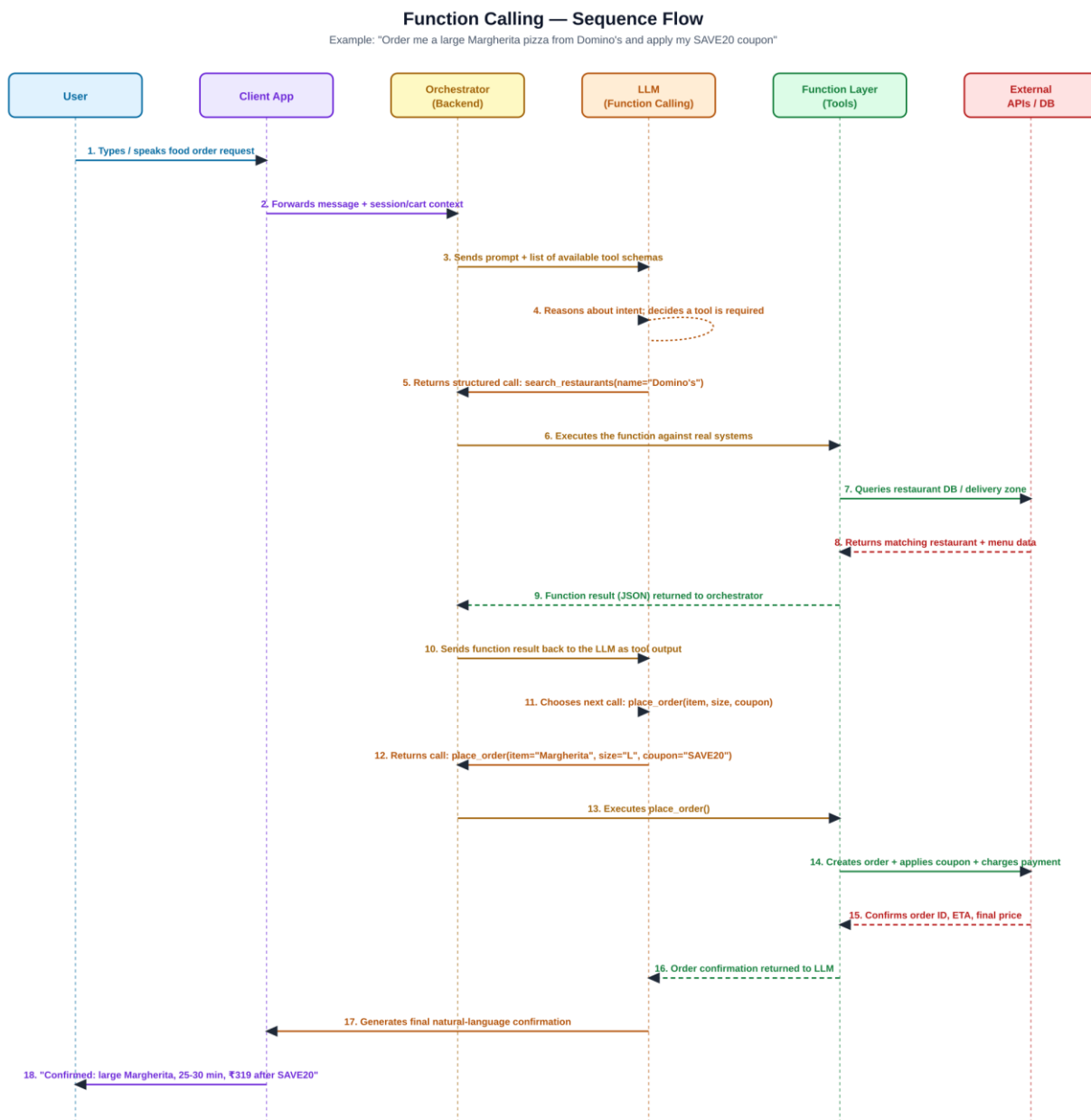


Figure 2 — Sequence flow for “Order me a large Margherita pizza from Domino’s and apply my SAVE20 coupon”

7.1 Step-by-Step Explanation

1. The user sends a natural-language request through the client app.
1. The orchestrator forwards the message, conversation history and tool schemas to the LLM.
2. The LLM determines a tool call is required and returns a structured `search_restaurants` call instead of prose.
3. The orchestrator executes the function against the restaurant/menu database and delivery-zone service.
4. The function's JSON result is appended to the conversation and sent back to the LLM.
5. The LLM inspects the result, then emits a second call — `place_order` — with the resolved restaurant, item, size and coupon code.
6. The orchestrator executes `place_order`, which validates the coupon, charges payment and creates the order in the OMS.
7. The confirmation (order ID, ETA, final price) flows back through the LLM, which drafts a short, friendly natural-language reply for the user.

8. Worked Example — Request & Response Payloads

8.1 Model's First Tool Call (turn 1)

```
// LLM response (tool_use block)
{
  "type": "tool_use",
  "name": "search_restaurants",
  "input": { "query": "Domino's", "lat": 12.971, "lng": 79.958 }
}
```

8.2 Application Executes & Returns Result

```
// tool_result sent back to the model
{
  "type": "tool_result",
  "content": {
    "restaurant_id": "RST-1042",
    "name": "Domino's Pizza - OMR",
    "eta_minutes": 28,
    "menu_hint": ["Margherita", "Pepperoni", "Farmhouse"]
  }
}
```

8.3 Model's Second Tool Call (turn 2)

```
{
  "type": "tool_use",
  "name": "place_order",
  "input": {
    "restaurant_id": "RST-1042",
    "items": [{ "item_name": "Margherita", "size": "L", "quantity": 1 }],
    "coupon_code": "SAVE20",
    "payment_method": "upi"
  }
}
```

8.4 Final Natural-Language Reply

"Done! Your large Margherita from Domino's - OMR is confirmed with SAVE20 applied. Total: ₹319. Estimated delivery: 25-30 minutes. Order ID ORD-48213."

9. Benefits of Function Calling

- Grounded, real-time answers — responses reflect the live menu, live prices and live order status instead of stale training data.
- Safe separation of concerns — the model decides intent; the developer's code performs every real-world side effect, so business rules and auth checks stay enforceable.
- Composability — new capabilities (loyalty points, group ordering) are added simply by registering a new schema, with no retraining.
- Structured, reliable output — arguments are schema-validated JSON rather than free text that would need fragile parsing.
- Multi-step reasoning — the model can chain several tool calls to fulfil one compound user request.

10. Challenges & Limitations

- Argument hallucination — the model may fabricate a plausible-looking but wrong value (e.g. a coupon code); server-side validation of every function's inputs is mandatory.
- Latency — each extra tool-call round trip adds a network hop, so chained calls (search → order) increase response time.
- Irreversible actions — payment and order-placement functions should require an explicit confirmation step before execution to avoid unintended charges.
- Schema ambiguity — overlapping or vaguely described tools increase the chance the model calls the wrong one; schemas must stay narrow and well documented.
- Error handling — the orchestrator must catch function failures (timeouts, out-of-stock items) and feed a clear error back to the model so it can recover gracefully rather than stall.

11. Conclusion

Function calling is what turns a conversational LLM into a genuinely useful food-ordering agent: the model supplies natural-language understanding and multi-step reasoning, while a well-defined set of tool schemas keeps every real action — searching, ordering, paying, tracking — inside code the developer controls and can validate. The layered architecture in this document (client → orchestrator →

LLM → function layer → external systems) generalizes well beyond food ordering to any assistant that must act on real, live systems rather than just talk about them.

References

1. Anthropic — Claude Docs, “Tool use (function calling)”, docs.claude.com
2. OpenAI — “Function calling”, platform.openai.com/docs
3. JSON Schema Specification, json-schema.org